

JOIA GESTÃO INTELIGENTE

API de auditoria gerencial

Resposta à especificação da RDS
versão 2.0 · 22 de setembro de 2026

Rede Jolô Gelato · joiagest.com.br
Acompanha o contrato `api-joia.openapi.yaml`

Documento único, para ser lido por quem pediu: **a especificação "Ampliação da API de leitura para auditoria gerencial RDS" foi atendida na Etapa 1**, e o que ela pede e ainda não existe está declarado aqui, sem maquiagem.

O levantamento foi feito direto no código do Joia e no banco de produção, sem alterar nada da operação das lojas. A camada analítica é **leitura** do que o sistema já grava.

- **Parte I — a API:** endereço, chave, todos os caminhos e o dicionário completo de campos (tipo, nulidade, unidade, arredondamento, origem e a fórmula de cada número), fuso, paginação e erros.
- **Parte II — o que ainda não existe:** requisito por requisito, as sete decisões de negócio que dependem de vocês e o plano em quatro etapas.

O contrato formal, em OpenAPI 3.1, vai no arquivo `api-joia.openapi.yaml`, que acompanha este documento e pode ser carregado direto numa ferramenta.

O que já está no ar, em cinco linhas

1. Endereço: `https://cevgkhndzpzvznwifhnm.supabase.co/functions/v1/joia-api`
2. Chave no cabeçalho: `Authorization: Bearer SUA_CHAVE` — a chave carrega a rede e, se quiser, uma unidade.
3. **Só GET.** Qualquer outro método responde 405: esta porta lê, não escreve.
4. **20 caminhos:** 9 consolidados (totais prontos) e 11 analíticos (registro a registro, paginados, com ordem determinada).
5. Provado em 22/09/2026: os onze caminhos novos com dado real; chave de uma unidade pedindo outra responde a dela, com zero registro; chave de outra rede não enxerga nada desta.

Exemplo, de ponta a ponta:

```
curl -H "Authorization: Bearer SUA_CHAVE" \
  "https://cevgkhndzpzvznwifhnm.supabase.co/functions/v1/joia-api/pedidos?de=2026-09-01&ate=2026-09-22&por_pagina=200&pagina=1"
```

PARTE I — A API

I.1. Regras gerais

I.1.1 Endereço, método e chave

```
GET https://cevgkhndzpzvznwifhnm.supabase.co/functions/v1/joia-api/<caminho>
Authorization: Bearer SUA_CHAVE
```

Só `GET`. Qualquer outro método responde **405** — a API lê, não escreve. `OPTIONS` responde só o CORS.

A chave decide o alcance, não o parâmetro:

- a chave carrega a **rede** (`loja_id`); não há como pedir outra;
- se a chave carrega **unidade** (`sucursal_id`), o parâmetro `loja` é ignorado e a resposta devolve a unidade da chave. Provado em 22/09/2026: chave presa a Alphaville pedindo `loja=suc_mt1unhbx2xrb` respondeu com a unidade dela e zero registros.

I.1.2 Parâmetros comuns

parâmetro	tipo	padrão	observação
<code>de</code>	<code>AAAA-MM-DD</code>	30 dias atrás	começo do período, inclusive
<code>ate</code>	<code>AAAA-MM-DD</code>	hoje	fim do período, inclusive
<code>loja</code>	<code>suc_...</code>	rede toda	ignorado quando a chave já tem unidade
<code>pagina</code>	inteiro ≥ 1	1	só nos caminhos analíticos
<code>por_pagina</code>	inteiro 1–1000	200	acima de 1000 é cortado em 1000

`de` e `ate` são comparados **inclusive** (`between`). Um dia só é `de=ate=AAAA-MM-DD`.

I.1.3 Fuso e corte do dia

- **Venda** (`/faturamento`, `/produtos`, `/pagamentos`, `/pedidos`, `/itens`, `/pagamentos-analitico`): a data sai de `data_venda at time zone 'America/Sao_Paulo'`. Venda das 23h é **do dia dela**, não do dia seguinte em UTC.
- **Estoque, produção, contagem e financeiro**: a data já é gravada pelo Joia como dia local (`date`), sem hora. Não há conversão — e não há como haver erro de fuso.
- `gerado_em`, `criado_em` e qualquer `timestamp` viajam em **UTC**, no formato ISO-8601 com `Z`. Converter para São Paulo é do lado de quem lê.
- O padrão de `de / ate` (30 dias) é calculado em UTC–3.

I.1.4 Arredondamento — a regra por natureza do número

natureza	casas	onde
dinheiro	2	<code>valor</code> , <code>total</code> , <code>valor_total</code> , <code>taxa_calculada</code> , <code>valor_ajuste</code> ...
preço unitário de venda	4	<code>preco_unitario</code>
quantidade de estoque	4	<code>quantidade</code> , <code>previsto</code> , <code>produzido</code> , <code>saldo_*</code>
saldo do painel de estoque	3	<code>/estoque.saldo</code> , <code>minimo</code>
custo unitário	4 a 6	<code>custo_medio</code> (4); <code>custo_unitario</code> e <code>custo</code> (6)

Arredondamento é `round` do Postgres (meio para cima, em `numeric` — sem ponto flutuante binário, sem centavo perdido). **Toda soma é feita antes de arredondar**; o arredondamento é a última operação.

I.1.5 Nulo, vazio e zero

Três coisas diferentes, e a API não as confunde:

- `null` = o Joia não tem esse dado para esse registro (ex.: `pagamento` de um título ainda em aberto);
- `" "` ou `"-"` = o campo existe e está em branco no cadastro;
- `0` = o número é zero de verdade.

Todo campo monetário calculado usa `coalesce(...,0)`: some sem medo, não vem `null` no meio de uma conta.

I.1.6 Envelope e paginação

Os caminhos analíticos devolvem sempre:

```
{
  "api-versao": "2.0",
  "gerado-em": "2026-09-22T18:40:11.204Z",
  "periodo": { "de": "2026-09-01", "ate": "2026-09-22" },
  "loja": "suc_mtlnpcg7b3m3",
  "pagina": {
    "pagina": 1, "por_pagina": 200, "nesta_pagina": 200,
    "total_registros": 1435, "total_paginas": 8
  },
  "avisos": ["..."],
  "pedidos": [ ... ]
}
```

campo do envelope	significado
<code>api-versao</code>	versão do contrato (<code>2.0</code>). Muda quando um campo muda de sentido
<code>gerado-em</code>	instante da resposta, UTC
<code>periodo</code>	o período efetivamente aplicado (já com os padrões resolvidos)
<code>loja</code>	a unidade aplicada, ou <code>"rede toda"</code>
<code>pagina.total_registros</code>	quantos registros o filtro tem no total , não na página
<code>avisos</code>	lista de ressalvas honestas; só aparece quando há o que ressaltar

Ordenação determinística em todos eles: a chave de ordem sempre termina num identificador único, então a página 2 nunca repete nem pula linha da página 1. A ordem de cada caminho está na tabela dele, abaixo.

Uma ressalva de paginação: `total_registros` viaja **dentro da linha** (`count(*) over()`). Numa página **além do fim** não vem linha nenhuma, e aí `total_registros` e `total_paginas` voltam `0`. Pagine pelo total lido na primeira página, e não interprete esse `0` como "não há dados".

Os caminhos `/plano-de-contas`, `/fichas` e `/reconciliacao/estoque` **não são paginados**: devolvem o conjunto inteiro (são cadastros e fechamentos, não razão). Os consolidados da v1 mantêm o formato antigo, sem envelope — nada neles mudou.

I.1.7 Erros

Toda resposta de erro tem o mesmo formato:

```
{ "erro": "A data inicial é depois da final.",  
  "api-versao": "2.0", "gerado-em": "2026-09-22T18:40:11.204Z" }
```

código	quando
400	data fora de AAAA-MM-DD ; de depois de ate ; data diferente de emissao vencimento pagamento em /titulos
401	sem chave, chave inválida, ou chave desativada (ativa=false)
404	caminho que não existe — a resposta já traz a lista dos válidos
405	qualquer método que não seja GET
500	falha ao consultar o banco; vem com detalhe

Não há 403: quem não pode ver uma rede simplesmente **não a vê** — a chave filtra na origem, e a resposta sai vazia, nunca com dado de outra rede.

I.1.8 Por dentro, e o que foi conferido

As 18 funções `api_*` são `stable security definer`, só `select`, com `search_path` fixo em `public`. A permissão de executar é **só** de `postgres` e `service_role` — conferido campo a campo em 22/09/2026: `anon` e `authenticated` não alcançam nenhuma delas. Quem chega pelas chaves públicas do projeto não chega aqui; só a função de borda chega.

A prova dos onze caminhos novos foi feita de dentro do próprio banco (o ambiente de desenvolvimento não alcança `*.supabase.co`), com a extensão `pg_net` instalada **para a prova e removida depois**: a porta de saída HTTP do banco não fica aberta sem necessidade.

I.2. Caminhos analíticos

I.2.1 GET /pedidos — venda a venda

Ordem: `data_venda`, `numero`, `pedido_ref`. Uma linha por pedido, **inclusive os cancelados** (marcados) — auditoria precisa vê-los.

campo	tipo	nulo?	unidade / casas	de onde vem
<code>pedido_ref</code>	texto	não	—	<code>pedidos.ref_local</code> , identificador estável do pedido
<code>unidade</code>	texto	sim	—	<code>sucursais.ref_local</code>
<code>unidade_nome</code>	texto	sim	—	<code>sucursais.nome</code>
<code>numero</code>	inteiro	sim	—	número do pedido na unidade
<code>data_venda</code>	data	não	dia da loja	<code>data_venda</code> convertida para São Paulo

campo	tipo	nulo?	unidade / casas	de onde vem
hora	texto HH:MM	sim	hora da loja	gravada pelo PDV
criado_em	timestamp	sim	UTC	quando a linha nasceu no banco
tipo	texto	sim	—	balcão / mesa / delivery / retirada
canal	texto	sim	—	canal de venda
origem_venda	texto	sim	—	de onde o pedido entrou
fase	texto	sim	—	situação do pedido (entregue , cancelado ...)
cliente	texto	sim	—	cliente_nome
caixa_ref	texto	sim	—	o caixa em que a venda foi feita
operador	texto	sim	—	operador do caixa (o Joia não grava operador por pedido)
mesa	inteiro	sim	—	número da mesa
comanda	texto	sim	—	nome da comanda
valor_bruto	decimal	não	R\$, 2	total + desconto + cupom - taxa_entrega - taxa_servico
desconto	decimal	não	R\$, 2	desconto do pedido
cupom	decimal	não	R\$, 2	valor do cupom aplicado
taxa_entrega	decimal	não	R\$, 2	frete cobrado
taxa_servico	decimal	não	R\$, 2	os 10% quando houver
valor_liquido	decimal	não	R\$, 2	pedidos.total — é o que o cliente pagou
cancelado	booleano	não	—	fase = 'cancelado'
cancelado_em	texto	sim	dia+hora da loja	de cancelamentos
cancelado_motivo	texto	sim	—	motivo escolhido no cancelamento
cancelado_por	texto	sim	—	quem autorizou
estoque_voltou	booleano	sim	—	se o cancelamento devolveu o estoque

Faturamento = soma de `valor_liquido` **onde** `cancelado = false`. É exatamente o que `/faturamento` devolve; os dois batem por construção.

1.2.2 GET /itens — item a item

Ordem: `data_venda, numero, item_ref`.

campo	tipo	nulo?	unidade / casas	de onde vem
item_ref	texto	não	—	pedido_itens.ref_local
pedido_ref	texto	não	—	liga com /pedidos

campo	tipo	nulo?	unidade / casas	de onde vem
unidade	texto	sim	—	unidade da venda
numero	inteiro	sim	—	número do pedido
data_venda	data	não	dia da loja	—
produto	texto	sim	—	nome no momento da venda
quantidade	decimal	sim	unidade do produto	como foi vendido
preco_unitario	decimal	não	R\$, 4	preço praticado na venda
valor_total	decimal	não	R\$, 2	total da linha, como gravado
adicionais	JSON	não	—	lista de adicionais: nome, preço e ficha
observacao	texto	sim	—	o que o cliente pediu
pedido_cancelado	booleano	não	—	o pedido foi cancelado

Aviso que a resposta carrega: desconto por item, cortesia e cancelamento de item **não são gravados** pelo Joia. Quem quiser desconto por item precisa rateá-lo pelo desconto do pedido — e saber que é rateio.

I.2.3 GET /pagamentos-analitico — pagamento a pagamento

Ordem: data_venda, numero, pagamento_ref.

campo	tipo	nulo?	unidade / casas	de onde vem
pagamento_ref	texto	não	—	identificador da transação no Joia
pedido_ref	texto	não	—	liga com /pedidos
unidade, numero, data_venda	—	—	—	do pedido
forma	texto	não	—	nome da forma; "-" se a forma sumiu do cadastro
forma_tipo	texto	sim	—	dinheiro / crédito / débito / pix / voucher
bandeira	texto	sim	—	do cadastro da forma , não da transação
valor	decimal	não	R\$, 2	valor pago nessa forma
taxa_pct	decimal	não	%, do cadastro	taxa percentual da forma
taxa_fixa	decimal	não	R\$, do cadastro	taxa por transação
taxa_calculada	decimal	não	R\$, 2	$\text{valor} \times \text{taxa_pct} \div 100 + \text{taxa_fixa}$
valor_liquido_previsto	decimal	não	R\$, 2	$\text{valor} - \text{taxa_calculada}$
dias_recebimento	inteiro	não	dias corridos	do cadastro da forma
data_prevista	data	não	dia da loja	$\text{data_venda} + \text{dias_recebimento}$ (dias corridos , sem feriado)
equipamento	texto	sim	—	maquininha informada no PDV

campo	tipo	nulo?	unidade / casas	de onde vem
pedido_cancelado	booleano	não	—	—

Aviso que a resposta carrega: taxa, prazo e bandeira saem do **cadastro de hoje**, não do retorno da adquirente. O Joia não conversa com maquininha: não existe data efetiva de recebimento nem estorno por transação. `valor_liquido_previsto` é previsão, e o nome diz isso.

1.2.4 GET /movimentacoes — o razão do estoque

Ordem: `data, hora, movimento_ref, linha`. Uma linha por item dentro de cada movimento.

campo	tipo	nulo?	unidade / casas	de onde vem
movimento_ref	texto	não	—	identificador do movimento
linha	inteiro	não	—	posição do item dentro do movimento (1, 2, 3...)
unidade	texto	sim	—	unidade da loja
data	data	não	dia da loja	—
hora	texto <code>HH:MM</code>	sim	hora da loja	—
criado_em	timestamp	sim	UTC	—
item_ref	texto	sim	—	insumo ou ficha estocável
item	texto	sim	—	nome no momento do movimento
unidade_medida	texto	não	—	<code>kg</code> , <code>un</code> , <code>L</code> ... (padrão <code>un</code>)
direcao	texto	não	—	<code>entrada</code> ou <code>saida</code> (padrão <code>saida</code>)
quantidade	decimal	não	unidade do item, 4	sempre positiva ; o sinal é <code>direcao</code>
custo_unitario	decimal	não	R\$, 6	custo do item no momento
valor_total	decimal	não	R\$, 2	<code>quantidade × custo_unitario</code>
origem	texto	não	—	a origem crua , para conferência
motivo	texto	não	—	motivo cadastrado, quando houver
classe	texto	não	—	classificação derivada — tabela abaixo
documento	texto	sim	—	nota, pedido ou identificação
observacao	texto	sim	—	—
contagem_sistema	decimal	sim	unidade do item	só em movimento de contagem
contagem_conferido	decimal	sim	unidade do item	só em movimento de contagem

Como `classe` é derivada (de `origem` + `direcao`):

origem	direção	classe
nota	qualquer	compra/entrada
venda	qualquer	consumo por venda
producao	entrada	produção

origem	direção	classe
producao	saída	consumo por produção
contagem	entrada	ajuste positivo
contagem	saída	ajuste negativo
transferencia, pedbase_saida, pedbase_entrada	entrada	transferência recebida
idem	saída	transferência enviada
qualquer outra	entrada	entrada avulsa
qualquer outra	saída	baixa identificada

Aviso que a resposta carrega: a classe é derivada — a `origem` crua vai junto para conferência. Saldo anterior e posterior **por movimento** não são gravados; a reconciliação vem de `/reconciliacao/estoque`.

I.2.5 GET /inventarios — contagem item a item

Ordem: `data, contagem_ref, item`.

campo	tipo	nulo?	unidade / casas	de onde vem
contagem_ref	texto	não	—	identificador da contagem
unidade	texto	sim	—	—
data	data	não	dia da loja	o dia contado
hora	texto	sim	—	—
retroativa	booleano	não	—	contagem lançada para um dia anterior
lançada_em	data	sim	dia da loja	quando foi efetivamente lançada
movimento_ref	texto	sim	—	o movimento de ajuste gerado
item_ref, item, unidade_medida	—	—	—	o item contado
saldo_sistema	decimal	não	unidade do item, 4	o que o Joia dizia ter
quantidade_contada	decimal	não	unidade do item, 4	o que a loja contou
diferenca	decimal	não	unidade do item, 4	<code>contado - sistema</code> (negativo = falta)
custo	decimal	não	R\$, 6	custo unitário na hora da contagem
valor_ajuste	decimal	não	R\$, 2	<code>diferenca × custo</code>
sentido	texto	não	—	<code>ajuste positivo</code> / <code>ajuste negativo</code> / <code>sem diferença</code>

A divergência entre o sistema e o físico mora aqui — é este o número de perda por conferência, não o de `/reconciliacao/estoque`.

I.2.6 GET /producao-analitico — produção item a item

Ordem: data, numero, produto.

campo	tipo	nulo?	unidade / casas	de onde vem
ordem_ref	texto	não	—	identificador da ordem
numero	texto	sim	—	número da ordem
unidade, data, hora	—	—	dia/hora da loja	—
situacao	texto	sim	—	situação da ordem
responsavel	texto	sim	—	quem produziu
movimento_ref	texto	sim	—	o movimento de estoque gerado
ficha_ref	texto	sim	—	a ficha produzida
produto	texto	sim	—	nome do produto
destino	texto	sim	—	onde o produzido entrou
unidade_medida	texto	não	—	padrão kg
previsto	decimal	não	unidade do item, 4	o que a ordem previa
produzido	decimal	não	unidade do item, 4	o que saiu de verdade (pesado)
diferenca	decimal	não	unidade do item, 4	produzido – previsto
receitas	decimal	não	—	quantas receitas foram multiplicadas

Aviso que a resposta carrega: insumo previsto × insumo efetivamente baixado, item a item, **não é gravado** pela ordem de produção. O que se sabe é o consumo total pela origem producao em /movimentacoes.

I.2.7 GET /titulos — contas a pagar e a receber

Parâmetro próprio: data=emissao|vencimento|pagamento (padrão vencimento) — escolhe qual data o período filtra. A ordem é sempre vencimento, titulo_ref.

campo	tipo	nulo?	unidade / casas	de onde vem
titulo_ref	texto	não	—	identificador do lançamento
unidade	texto	sim	—	—
tipo	texto	sim	—	receita, despesa ou transferencia
descricao	texto	sim	—	—
documento	texto	sim	—	número da nota / documento
fornecedor	texto	sim	—	nome do fornecedor
categoria / subcategoria	texto	sim	—	o plano de contas, dois níveis
emissao	data	sim	dia da loja	hoje é a data mais próxima de competência
vencimento	data	sim	dia da loja	—
pagamento	data	sim	dia da loja	null enquanto em aberto

campo	tipo	nulo?	unidade / casas	de onde vem
valor_original	decimal	não	R\$, 2	valor antes de juros e multa
juros / multa	decimal	não	R\$, 2	—
valor	decimal	não	R\$, 2	valor efetivo do título
pago	booleano	não	—	—
cancelado	booleano	não	—	título cancelado continua aparecendo, marcado
conciliado	booleano	não	—	bateu com o extrato
data_conciliacao	data	sim	—	—
conta	texto	sim	—	conta de capital usada
forma_pagamento	texto	sim	—	—
origem / origem_ref	texto	sim	—	o que gerou o título (ex.: nota de entrada)
criado_em	timestamp	sim	UTC	—

Aviso que a resposta carrega: não existe campo de **competência** nem **centro de resultado**; a emissão é a data mais próxima da competência. Pagamento parcial e saldo em aberto também não são gravados — o título é pago ou não é.

1.2.8 GET /extrato — o que entrou e saiu de caixa e banco

Junta **duas** origens: lançamentos financeiros pagos e movimentos de caixa (sangria, suprimimento, transferência). Ordem: `data, hora, ref`.

campo	tipo	nulo?	unidade / casas	de onde vem
lançamento_ref	texto	não	—	identificador da linha
unidade	texto	sim	—	—
data	data	não	dia da loja	data do pagamento (título) ou do movimento
hora	texto	sim	—	<code>null</code> nos títulos; preenchido no caixa
conta	texto	sim	—	conta de capital, ou "Caixa da loja"
classe	texto	não	—	recebimento, pagamento, transferência entre contas, sangria, suprimimento
sentido	texto	não	—	entrada ou saída
valor	decimal	não	R\$, 2	sempre positivo ; o sinal é <code>sentido</code>
descricao	texto	sim	—	descrição do título ou motivo da sangria
categoria	texto	sim	—	só nos títulos
título_ref	texto	sim	—	liga com <code>/titulos</code>
conciliado	booleano	não	—	<code>false</code> nos movimentos de caixa
responsavel	texto	sim	—	só nos movimentos de caixa

Só entram títulos **pagos e não cancelados**, filtrados pela data de pagamento.

Aviso que a resposta carrega: transferência entre contas vem com classe própria — **não é receita nem despesa**, e somá-la ao faturamento é erro. Saldo anterior/posterior por conta não é gravado; o saldo inicial de cada conta está em `contas_capital`.

1.2.9 GET /plano-de-contas — sem paginação

campo	tipo	nulo?	de onde vem
<code>nivel</code>	inteiro	não	<code>1</code> = categoria, <code>2</code> = subcategoria
<code>categoria_ref</code> / <code>categoria</code>	texto	não	<code>categorias_financeiras</code>
<code>tipo</code>	texto	sim	<code>receita</code> ou <code>despesa</code>
<code>subcategoria_ref</code> / <code>subcategoria</code>	texto	sim	<code>null</code> no nível 1
<code>ordem</code>	inteiro	sim	posição na tela

Aviso: dois níveis apenas. Código contábil, natureza, posição na DRE e vigência **não existem** no cadastro.

1.2.10 GET /fichas — sem paginação

Parâmetro próprio: `ficha=fi_...` traz uma só. Uma linha por **ingrediente** da ficha; a ficha sem ingrediente aparece com os campos de ingrediente em branco.

campo	tipo	nulo?	unidade / casas	de onde vem
<code>ficha_ref</code> / <code>ficha</code> / <code>codigo</code>	texto	—	—	cabeçalho da ficha
<code>unidade</code>	texto	sim	—	unidade da ficha
<code>rendimento</code> / <code>rendimento_unidade</code>	decimal / texto	sim	—	quanto a receita rende
<code>unidades_venda</code>	decimal	sim	—	em quantas unidades de venda o rendimento se divide
<code>preco</code>	decimal	sim	R\$	preço de venda
<code>estocavel</code>	booleano	sim	—	se o produzido entra no estoque
<code>destino</code>	texto	sim	—	para onde o produzido vai
<code>ingrediente_ref</code> / <code>ingrediente</code>	texto	sim	—	insumo ou subficha
<code>quantidade</code>	decimal	sim	unidade do ingrediente	quanto a receita usa
<code>unidade_ingrediente</code>	texto	sim	—	—
<code>perda</code>	decimal	sim	%	perda declarada na ficha
<code>subficha_ref</code>	texto	sim	—	preenchido quando o ingrediente é outra ficha
<code>custo_unitario</code>	decimal	sim	R\$	custo de hoje do insumo ou da subficha

Aviso que a resposta carrega, e é o mais importante deste documento: a ficha técnica **não tem vigência**. O que sai aqui é a receita de **hoje**. Calcular CPV de uma venda antiga com esta ficha recalcula o passado com a receita atual. A saída é congelar o CPV na venda — decisão 2 do a Parte II deste documento.

1.2.11 GET /reconciliacao/estoque — sem paginação

Uma linha por item e unidade. A equação anunciada em **regra** :

$$\text{saldo_inicial} + \text{entradas} - \text{saidas} = \text{saldo_final}$$

Como cada ponta é obtida, sem rodeio:

- **saldo_final** = saldo de hoje em **estoque_unidade** **menos** tudo que se moveu **depois** de **ate** — isto é, o saldo rebobinado até o fim do período;
- **entradas** / **saidas** = soma das quantidades do razão **dentro** do período, por direção;
- **saldo_inicial** = **saldo_final** - **entradas** + **saidas**.

campo	tipo	nulo?	unidade / casas	o que é
unidade , item_ref , item , unidade_medida	texto	—	—	o item
saldo_inicial	decimal	não	unidade do item, 4	saldo na véspera de de
entradas / saidas	decimal	não	unidade do item, 4	somas do período
saldo_final	decimal	não	unidade do item, 4	saldo no fim de ate
consumo_venda	decimal	não	idem	saídas com origem venda
consumo_producao	decimal	não	idem	saídas com origem producao
producao_entrada	decimal	não	idem	entradas com origem producao
compras	decimal	não	idem	entradas com origem nota
transferencias_recebidas / transferencias_enviadas	decimal	não	idem	—
ajuste_positivo / ajuste_negativo	decimal	não	idem	vindos de contagem
baixas_identificadas	decimal	não	idem	saídas de qualquer outra origem

Aviso que a resposta carrega: a equação **fecha por construção** — o razão é a fonte do saldo, então ela não prova nada sozinha. O que prova divergência é a contagem física, em **/inventarios**. Este caminho serve para **abrir** o movimento do período por natureza, não para pegar erro.

I.3. Consolidados (v1) — campos

Não mudaram na v2.0 e continuam sem envelope, para não quebrar quem já lê.

caminho	campos
/lojas	ref_local, nome, cidade, uf, matriz, ativa
/faturamento	topo: total, pedidos, ticket_medio; dias[]: dia, pedidos, total, ticket, desconto, taxa_entrega — cancelado fora
/produtos	produto, quantidade, total (parâmetro limite, padrão 50, máx. 500)
/pagamentos	forma, quantidade, total — cancelado fora
/estoque	topo: itens, valor_total_em_estoque, abaixo_do_minimo, filtro; estoque[]: unidade_loja, item, unidade, saldo, custo_medio, valor, minimo, abaixo_do_minimo. Fotografia de agora — ignora de / ate; abaixo=1 filtra a lista, e os totais do topo continuam sendo do conjunto inteiro
/financeiro	topo: a_pagar{em_aberto,pago}, a_receber{em_aberto,recebido}; lancamentos[]: tipo, categoria, situacao, quantidade, total. Filtra por vencimento
/producao	dia, numero, situacao, previsto, produzido, diferenca
/contagens	dia, retroativa, itens_conferidos, sobra, perda (positiva), resultado
/resumo	venda + estoque + financeiro + produção + contagens numa resposta só

I.4. Como as pontas se ligam

de	para	pela chave
/itens	/pedidos	pedido_ref
/pagamentos-analitico	/pedidos	pedido_ref
/extrato	/titulos	titulo_ref
/inventarios	/movimentacoes	movimento_ref
/producao-analitico	/movimentacoes	movimento_ref
/titulos	/plano-de-contas	categoria + subcategoria
/movimentacoes, /reconciliacao/estoque	/fichas	item_ref = ficha_ref quando o item é ficha estocável
tudo	/lojas	unidade = ref_local

Conferências que fecham por construção (e servem de teste de fumaça):

- soma(/pedidos.valor_liquido onde não cancelado) = /faturamento.total
- soma(/pagamentos-analitico.valor onde não cancelado) = /pagamentos
- /reconciliacao/estoque: saldo_inicial + entradas - saidas = saldo_final
- /inventarios.diferenca × custo = /inventarios.valor_ajuste, e a soma por contagem bate com sobra / perda de /contagens

I.5. O que esta API não responde — e por quê

Repetido aqui de propósito, para não haver surpresa na homologação:

1. **CPV histórico confiável** — a ficha não tem vigência; o CPV precisa ser congelado na venda (decisão 2 do diagnóstico).
2. **Alteração retroativa** — não há `atualizado_em` na maioria das tabelas; não dá para detectar quem mudou o quê depois (Etapa 2).
3. **Cortesia** — não é classificação própria hoje (decisão 1).
4. **Competência e centro de resultado** — não existem no cadastro (decisões 3 e 5).
5. **Adquirente por transação** — o Joia não fala com a maquininha: bandeira e taxa saem do cadastro, e a data de recebimento é previsão.
6. **Patrimônio** (imobilizado, depreciação, empréstimos, folha) — não é assunto do Joia hoje; balanço gerencial não é possível (decisão 7).

O detalhe de cada uma dessas lacunas, e o que precisa da decisão de vocês, está na **Parte II** deste mesmo documento.

PARTE II — O QUE AINDA NÃO EXISTE

Nada aqui é limitação da API: é dado que o Joia **não grava** hoje. Nenhuma API inventa número, e este documento prefere a ressalva ao número bonito e errado.

Veredito curto: o pedido faz sentido e é o caminho certo. A maior parte do que a RDS quer **já estava gravada** no Joia e só não estava exposta — isso se resolveu na API, sem tocar na operação das lojas. Três coisas, porém, **não existem** hoje: ficha técnica com vigência, CPV congelado na venda e carimbo de alteração (`atualizado_em`). Sem elas, a auditoria histórica recalcula o passado com a receita de hoje.

II.1. Arquitetura atual, em uma página

- **Banco:** Supabase (Postgres), projeto `cevghkndzpzvnzweifhnm`. Uma rede (`loja_id`) com quatro unidades (`sucursal_id`): Matriz, Santa Fé do Sul, Alphaville e Jales.
- **ERP:** aplicação de página única que opera **offline-first**. Cada aparelho guarda tudo localmente e sincroniza; o banco é o ponto de encontro, não a fonte única em tempo real.
- **API atual:** função de borda `joia-api`, sem JWT, autenticada por chave própria (`api_chaves`, guardando só o sha-256). Ela chama funções SQL `api_*` marcadas `security definer`, revogadas para `anon` e `authenticated`. Só GET; qualquer outro método responde 405.
- **Chave:** presa a uma rede e, opcionalmente, a uma unidade. Quando a chave tem unidade, o parâmetro `loja` **não** a tira de lá.

Onde cada assunto mora

assunto	tabela	observação
venda	<code>pedidos</code>	cabeçalho; <code>fase</code> = entregue/cancelado
itens	<code>pedido_itens</code>	<code>opcoes</code> em JSON traz adicionais com preço e ficha
pagamentos	<code>pedido_pagamentos</code>	taxa e prazo moram em <code>formas_pagamento</code>
cancelamento	<code>cancelamentos</code>	motivo, operador, hora, se o estoque voltou
caixa	<code>caixas</code> + <code>caixa_movimentos</code>	conferência e fotografia do fechamento
estoque	<code>movimentacoes_estoque</code> (razão) + <code>estoque_unidade</code> (saldo)	linhas em JSON
contagem	<code>contagens_estoque</code>	sistema × contado × diferença, por item
produção	<code>ordens_producao</code>	previsto × real por item
receita	<code>fichas_tecnicas</code> + <code>ficha_itens</code>	sem vigência
financeiro	<code>lancamentos_financeiros</code>	emissão, vencimento, pagamento
plano de contas	<code>categorias_financeiras</code> + <code>subcategorias_financeiras</code>	dois níveis

assunto	tabela	observação
bancos e caixas	contas_capital	saldo inicial por conta

II.2. Requisito × situação atual

II.2.1 Atendido hoje

requisito	onde
somente leitura, só GET	joia-api responde 405 a qualquer outro método
autenticação por chave, com hash	api_chaves.chave_hash (sha-256)
revogação e registro de uso	ativa, ultimo_uso, usos
restrição por rede	a chave carrega loja_id
restrição opcional por unidade	sucursal_id na chave vence o parâmetro
funções SQL fora do alcance das chaves públicas	revoke para anon / authenticated
fuso da loja no corte do dia	venda das 23h é do dia dela
cancelado fora do faturamento	fase <> 'cancelado'
consolidados de venda, estoque, financeiro, produção e contagem	/faturamento, /produtos, /pagamentos, /estoque, /financeiro, /producao, /contagens, /resumo

II.2.2 Parcialmente atendido — o dado existe, falta expor (ou calcular)

requisito	o que já existe	o que falta
pedidos analíticos	número, tipo, canal, fase, cliente, caixa, total, taxa, desconto, cupom, taxa de serviço, data/hora, operador (via caixa), criado_em	expor linha a linha; separar bruto de líquido; juntar o cancelamento (mora em cancelamentos); não há updated_at
itens do pedido	nome, quantidade, unitário, total, adicionais (nome, preço, ficha), observação	desconto por item, cortesia, cancelamento do item, ficha usada, CPV — nada disso é gravado
pagamentos	forma, valor, equipamento; e no cadastro da forma: taxa_pct, taxa_fixa, dias_recebimento, bandeira	expor; calcular líquido e data prevista; não há data efetiva do recebimento nem estorno por transação
movimentações de estoque	data, hora, motivo, origem, item, quantidade, unidade, custo, direção, documento, unidade da loja	identificador por linha, saldo anterior/posterior, updated_at; e a separação "baixa identificada × ajuste" depende de classificar os motivos
contagens	por item: sistema, contado, diferença, custo, valor; ajuste positivo e negativo já somados em separado (ganho / perda); retroativa; movimento gerado	justificativa por item; aprovação/fechamento formal

requisito	o que já existe	o que falta
produção	por item: previsto, real, diferença, ficha, unidade, destino; movimento de estoque ligado	insumos previstos × efetivamente baixados, item a item; perda da produção separada
contas a pagar/receber	emissão, vencimento, pagamento, valor, juros, multa, valor original, pago, conciliado, fornecedor, categoria, documento, origem, cancelado	competência (hoje só existe emissão), centro de resultado, saldo e pagamento parcial, parcelamento explícito (hoje é "(1/3)" dentro do texto)
movimentações financeiras	<code>caixa_movimentos</code> (sangria, suprimento, transferência, com conta de destino) e os lançamentos pagos	um extrato único por conta, com saldo anterior/posterior; a transferência já é identificável e não é tratada como receita
plano de contas	categoria > subcategoria, com tipo receita/despesa	código, natureza, nível, posição na DRE, vigência, histórico

II.2.3 Não atendido — o dado não existe

requisito	consequência
ficha técnica com vigência	não há como saber qual receita valia na data da venda; recalculando o passado usa a ficha de hoje
CPV teórico gravado na venda	idem: o CPV histórico muda quando alguém corrige uma ficha
<code>updated_at</code> na maioria das tabelas	não dá para detectar alteração retroativa nem fazer sincronização incremental confiável
saldo anterior/posterior por movimento	a reconciliação de estoque tem de ser refeita somando tudo
cortesia como classificação própria	hoje a cortesia não se distingue de desconto ou de baixa
competência separada do vencimento	DRE por competência e fluxo por movimentação se confundem

requisito	consequência
centro de resultado	não existe no cadastro
patrimônio: imobilizado, depreciação, capital, empréstimos, aportes, retiradas, tributos a recolher, folha	balanço patrimonial gerencial não é possível hoje

II.2.4 Não aplicável / depende de terceiro

- **Adquirente e bandeira por transação:** o Joia não conversa com a maquininha. A bandeira existe no cadastro da forma de pagamento, não na venda. Só muda com integração de adquirente.
- **Data efetiva de recebimento de cartão:** idem — hoje só dá para projetar pela regra de dias do cadastro.
- **Ambiente de homologação:** existe um projeto só. Um segundo ambiente é decisão de custo.

II.3. Perguntas que precisam de decisão (Rafael / Ricardo-RDS)

1. **Cortesia** — como a loja registra hoje? Vai virar um motivo próprio de movimentação, uma marca no item da venda, ou as duas coisas?
2. **Ficha técnica** — prefere (a) **congelar** o CPV e a receita no momento da venda, ou (b) **versionar** o cadastro com vigência? Recomendo (a): resolve o histórico dali para frente, é muito mais simples e não muda a rotina de quem mexe na ficha.
3. **Competência** — adotar a data de emissão como competência, ou criar um campo próprio na tela do lançamento?
4. **Motivos de movimentação** — preciso da lista atual classificada em "baixa identificada", "ajuste", "consumo", "entrada" e "transferência". Sem isso a API não separa perda de ajuste com honestidade.
5. **Centro de resultado** — existe na operação de vocês? Se sim, em que nível (unidade, turno, praça)?
6. **Índice de perdas** — confirmam perdas ÷ CPV teórico como indicador principal, com perdas ÷ receita líquida só como complemento?
7. **Patrimônio** — o que virá da contabilidade e o que vocês querem lançar dentro do Joia?

II.4. Plano por etapas

Etapa 1 — só API, não encosta na operação · ENTREGUE em 22/09/2026

Os onze caminhos analíticos descritos na Parte I estão no ar, paginados, com ordem determinada, versão da API, carimbo de geração, erros padronizados, documentação e contrato OpenAPI — e as provas de segregação entre redes e entre unidades, feitas e registradas.

Um item da lista original não foi entregue, de propósito: `/cpv-teorico`. Um CPV calculado com a ficha de hoje e aplicado a uma venda antiga é um número que muda sozinho quando alguém corrige a receita — e como relatório de auditoria ele engana mais do que ajuda. Em lugar dele está `/fichas`, que entrega a receita e o custo **de hoje**, declarando isso: quem quiser calcular CPV calcula, sabendo exatamente o que está usando. O CPV confiável vem da decisão 2, na Etapa 3.

Nada aqui muda regra do Joia nem a rotina das lojas.

Etapa 2 — banco, sem mexer na regra (*risco baixo/médio*)

`atualizado_em` nas tabelas que a auditoria precisa vigiar, com gatilho; índices para as consultas novas. Não altera valor nenhum.

Etapa 3 — mexe no Joia (*risco médio, passa pelo portão, com guardião*)

Gravar na venda a ficha usada e o CPV teórico do item; cortesia como classificação própria; competência no lançamento financeiro; classificação dos motivos de movimentação. **Só depois das respostas da seção II.3.**

Etapa 4 — reconciliações fechadas e dashboards

Só depois que os números estiverem homologados.

II.5. Riscos

- **Etapa 1 é segura.** É leitura de dado que já existe; se algo sair errado, sai errado num relatório, não no caixa da loja.
- **Etapa 3 é a que exige cuidado:** toca o PDV e o financeiro, que operam todo dia nas lojas. Entra pelo portão de publicação, com teste-guardião próprio, como toda alteração deste sistema.
- **O risco de não fazer a Etapa 3** é a auditoria homologar números que mudam sozinhos quando alguém corrige uma ficha técnica antiga.
- **Não inventar:** enquanto o CPV histórico for calculado com a ficha de hoje, a resposta da API vai dizer isso, em campo próprio. Número com ressalva é melhor do que número bonito e errado.

Como seguir

A Etapa 1 está entregue e provada. A Etapa 2 (carimbo de alteração e índices) não altera valor nenhum e pode começar assim que vocês quiserem. A **Etapa 3 é a que depende das sete decisões da seção II.3** — ela toca o PDV e o financeiro, que operam todo dia nas lojas, e por isso passa pelo portão de publicação do Joia, com teste-guardião próprio, como toda alteração deste sistema.

Joia Gestão Inteligente · joiagest.com.br · 22/09/2026